Lab 07 — Ensemble Models, Built from Scratch

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 07 · Topic 6. Lecture: `slides/07_ensembles.pdf`. Demo notebooks in `notebooks/07-ensembles/`: `01_decision_boundaries.ipynb`, `02_bagging_forests.ipynb`, `03_boosting.ipynb` and `04_stacking_and_choice.ipynb`.

Notebook for this lab: `notebooks/07-ensembles/lab07_ensembles.ipynb`, with helpers in `lab07_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab07_ensembles_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first.

The rule of this lab. An ensemble is a loop around a weak learner. You write the loops with NumPy (the weak learner is a scikit-learn tree) and compare each with its scikit-learn twin on the **same split**. The notebook prints the difference; explain it (the random draws, the factor 2 of β, the folds).

In class (60 minutes): Parts A, B, C and D, the vote, bagging, AdaBoost and gradient boosting. **At home: Parts E and F**, stacking and the challenge; the solution notebook explains them.

By the end of the lab you should be able to:

1. Combine members by a hard and a soft vote and say when a vote cannot help.

2. Draw bootstrap samples, bag trees, read the out-of-bag error and see what `max_features` changes.
3. Write AdaBoost with stumps: the weighted error, the member weight β_b and the weight update.
4. Fit gradient boosting by residuals, and choose the shrinkage and the number of trees on validation data.
5. Build a stack on out-of-fold predictions and show the leak of in-fold predictions.

Part	Task	twin	Time
A	The gallery on the moons: hard vote and soft vote	`VotingClassifier`	12 min
B	Bagging by hand: bootstrap, vote, out-of-bag error, `max_features`	`BaggingClassifier`, `RandomForestClassifier`	15 min
C	AdaBoost with stumps: weights, β_b, the final vote	`AdaBoostClassifier`	20 min
D	Gradient boosting: residuals, shrinkage, early stopping	`GradientBoostingRegressor`	13 min
E	Stacking with out-of-fold predictions and the leak	`StackingClassifier`	home
F	Challenge: pseudo-residuals with `jax.grad`	`HistGradientBoostingClassifier`	home

Setup

```
cd faa
source venv/bin/activate        # see README: make venv, or pip install .
jupyter lab notebooks/07-ensembles/lab07_ensembles.ipynb
```

The data: the **moons** (300 points, noise 0.3, the running example of the lecture,

split 70/30), 20-feature synthetic data for `max_features`, a **noisy sine** (80 training, 40 validation points; the test targets are the noiseless sine on a longer range) and **heart disease** (303 patients, `datasets/`, read with polars). The helpers split the data and draw the boundary panels; the loops are yours.

Part A — The Gallery and a Vote

Three members of different families (logistic regression, 15-NN and an RBF SVM) draw three boundaries on the moons. A **hard vote** takes the majority label; a **soft vote** averages the probabilities and thresholds at 0.5.

A1. Hard vote and soft vote

Implement `hard_vote(labels)` (shape `(B, n)`, one row of 0/1 labels per member; ties go to class 1) and `soft_vote(probas)` (the mean probability and the label). The driver compares with `VotingClassifier(voting="soft")` and draws the boundaries.

1. What are the test accuracies of the members and of the votes? Is the vote better than the best member, than the mean member?
2. On how many test points do the members disagree, and why does a vote need that number to be above zero?
3. Replace the three members by three copies of 15-NN. What does the vote give, and what does it say about diversity?

Part B — Bagging by Hand

A bootstrap sample draws n examples with replacement: about 63.2 % of the distinct examples are in, 36.8 % are **out of bag**.

B1. Bootstrap, bagging, OOB error

Implement `bootstrap_indices(n, rng)`, `fit_bagging(X, y, n_trees, seed, max_features)` (one `DecisionTreeClassifier` per sample, and which examples each tree saw), `predict_bagging(trees, X)` (the mean probability) and `oob_error(trees, inbag, X, y)` (each example predicted only by the trees that did not see it). The driver compares 100 of your trees with `BaggingClassifier` and `RandomForestClassifier`, then sweeps `max_features` on 20

correlated features.

1. Is the unique share close to $1 - 1/e$? Why?
2. How do your OOB and test errors compare with the two twins? Why do they not agree to the last digit?
3. Which `max_features` has the lowest OOB error, and how much better is it than plain bagging?

Part C — AdaBoost with Stumps

Boosting trains the members in turn. Each stump is fitted with example weights; the examples it gets wrong weigh more in the next round.

C1. The loop

Implement `fit_adaboost(X, y, n_rounds)`: weights start at $1/n$; each round fits a stump with `sample_weight`, computes err_b, $\beta_b = \frac{1}{2} \ln \frac{1-\text{err}_b}{\text{err}_b}$ and the new weights $w_i^{(b+1)} \propto w_i^{(b)}\, e^{-\beta_b s_i \hat{s}_b(x_i)}$. Implement `predict_adaboost(stumps, betas, X)`. The driver reproduces the ten-point example of the lecture and compares 50 rounds on the moons with `AdaBoostClassifier`.

1. In the ten-point example, which points gain weight after round 1, and by how much? Is the final vote right on all ten?
2. How close is your ensemble to `AdaBoostClassifier`? The library reports β twice as large as yours: why does the vote not change?
3. Does the test error keep falling with more rounds? What would you use to choose the number of rounds?

Part D — Gradient Boosting for Regression

Gradient boosting adds trees that fit the **residuals**: $F_b = F_{b-1} + \eta\, \hat{f}_b$, a gradient step in the space of functions.

D1. Residuals, shrinkage, early stopping

Implement `fit_gbr(X, y, n_trees, eta, depth)`, `predict_gbr(model, X, n_trees)` and `best_rounds(model, X_val, y_val)` (the number of trees with the lowest validation RMSE). The driver compares with `GradientBoostingRegressor` and sweeps η.

1. How close is your model to `GradientBoostingRegressor`? Why can they agree to rounding error?
2. How does the best number of trees change with η, and what happens to the test RMSE at 300 trees when η is 1?
3. What does the model predict outside the training range, and why?

Part E — Stacking with Out-of-Fold Predictions

A **stack** feeds the predictions of the members to a meta-learner. In-fold predictions are too good (the member saw the point), so the meta-learner trusts the members too much: **leakage**.

E1. Out-of-fold features and the stack

Implement `out_of_fold(make_model, X, y, k, seed)`, `fit_stack(makers, X, y, k, seed, leak)` and `predict_stack(stack, X)`. The members are 1-NN, a tree and an SVM on the moons; the driver compares the leaking and the out-of-fold stack with `StackingClassifier`.

1. Which meta weights are large in the leaking stack and which in the out-of-fold stack? Which member does the leak over-trust?
2. Does the leak change the test accuracy a lot on the moons? Which number shows it most clearly?
3. How close is your out-of-fold stack to `StackingClassifier`?

Part F — Challenge: Pseudo-Residuals with `jax.grad`

For the logistic loss $\ell(F, y) = \ln(1 + e^F) - yF$ the negative gradient with respect to the score is $y - \sigma(F)$.

F1. Gradient boosting for classification

Implement `pseudo_residuals(y, F)` with `jax.grad` (not the formula), `fit_gbc(X, y, n_trees, eta, depth)` and `predict_gbc_proba(model, X)`. The driver checks the gradient against $y - \sigma(F)$ and compares with `HistGradientBoostingClassifier` and a random forest on heart disease.

1. How far is the `jax.grad` value from $y - \sigma(F)$, and why is it not exactly zero?

2. How close are the three models on 76 test patients? Can this split decide between them, and which protocol of Class 01 would you use?
3. **Project 1:** which learners (tree, KNN, network) would you put in a vote or a stack, and why?

Going further. Put the vote of Part A on heart disease with your three Project 1 learners, and compare it with its best member by repeated stratified cross-validation and the corrected paired t-test of Class 01.